

1 Methodology

This section describes the software architecture, USB Picture Transfer Protocol (PTP/ISO 15740) data acquisition pipeline, and system modeling for the Canon EOS R8 FastAPI control system.

1.1 System Data Flow Architecture

The system connects a web-based user interface to physical camera hardware via an asynchronous FastAPI backend and low-level `libgphoto2` PTP drivers. The architecture consists of four primary operational layers:

1. **Frontend UI (Browser):** HTML5/JavaScript dashboard managing live preview displays and user interaction controls.
2. **Backend API (FastAPI):** Asynchronous web application handling routing, CORS, HTTP response streams, and endpoint contracts.
3. **Controller Layer (CameraController):** Python thread-synchronized wrapper interfacing with `python-gphoto2` bindings and managing hardware locks.
4. **Hardware & Protocol Layer (libgphoto2 / USB PTP):** USB bulk endpoints transferring operational data packets to/from Canon EOS R8 SDRAM and CMOS sensor assemblies.

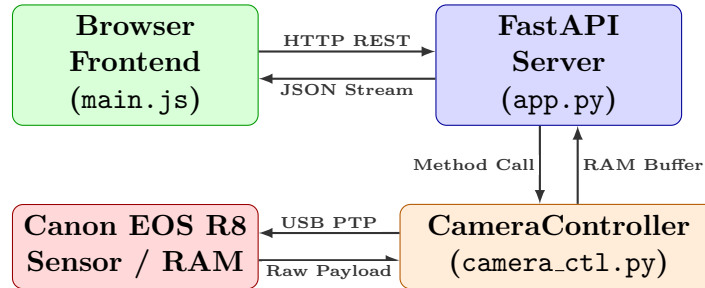


Figure 1: System architecture and data flow topology with enlarged controller block.

1.2 UML Use Case Diagram

Figure 2 illustrates the UML Use Case Diagram modeled after camera control software patterns. Actors are placed strictly outside the system boundary, and all use cases are cleanly separated into functional columns.

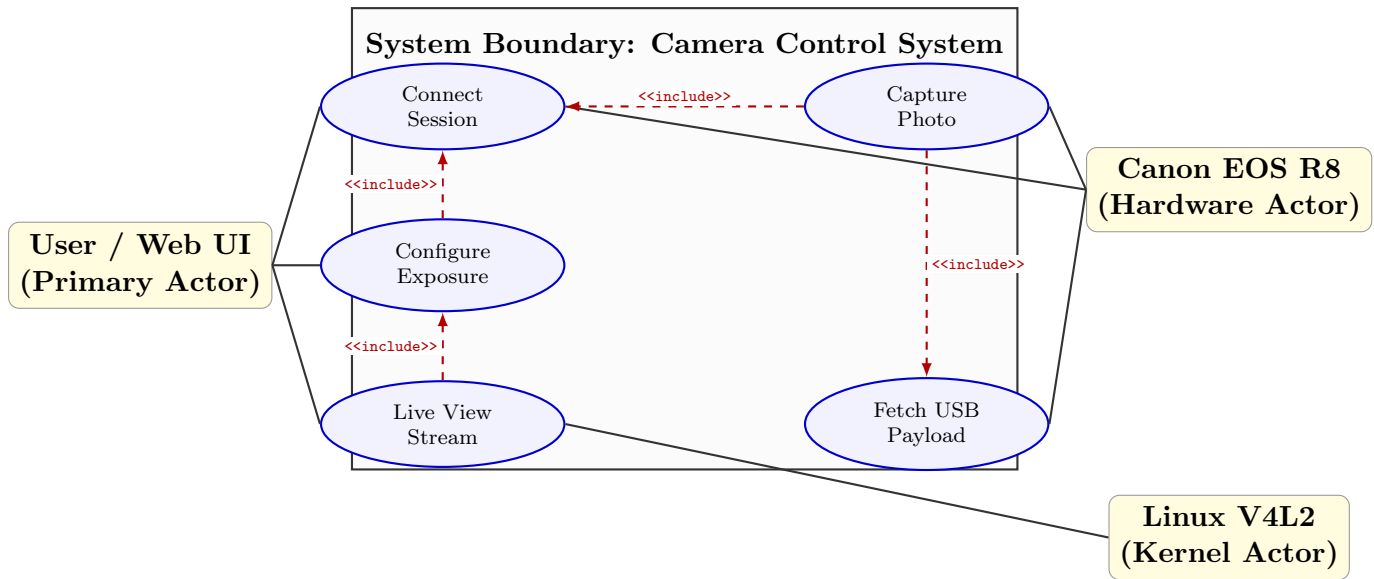


Figure 2: UML Use Case Diagram with clear actor association paths.

1.3 UML Sequence Diagram: Data Acquisition Lifecycle

Figure 3 details the **UML Sequence Diagram** for full-resolution data acquisition across all four system lifelines. All lifeline connections begin strictly at the bottom boundary of the entity header boxes, and side notes are offset to prevent lines from passing inside text.

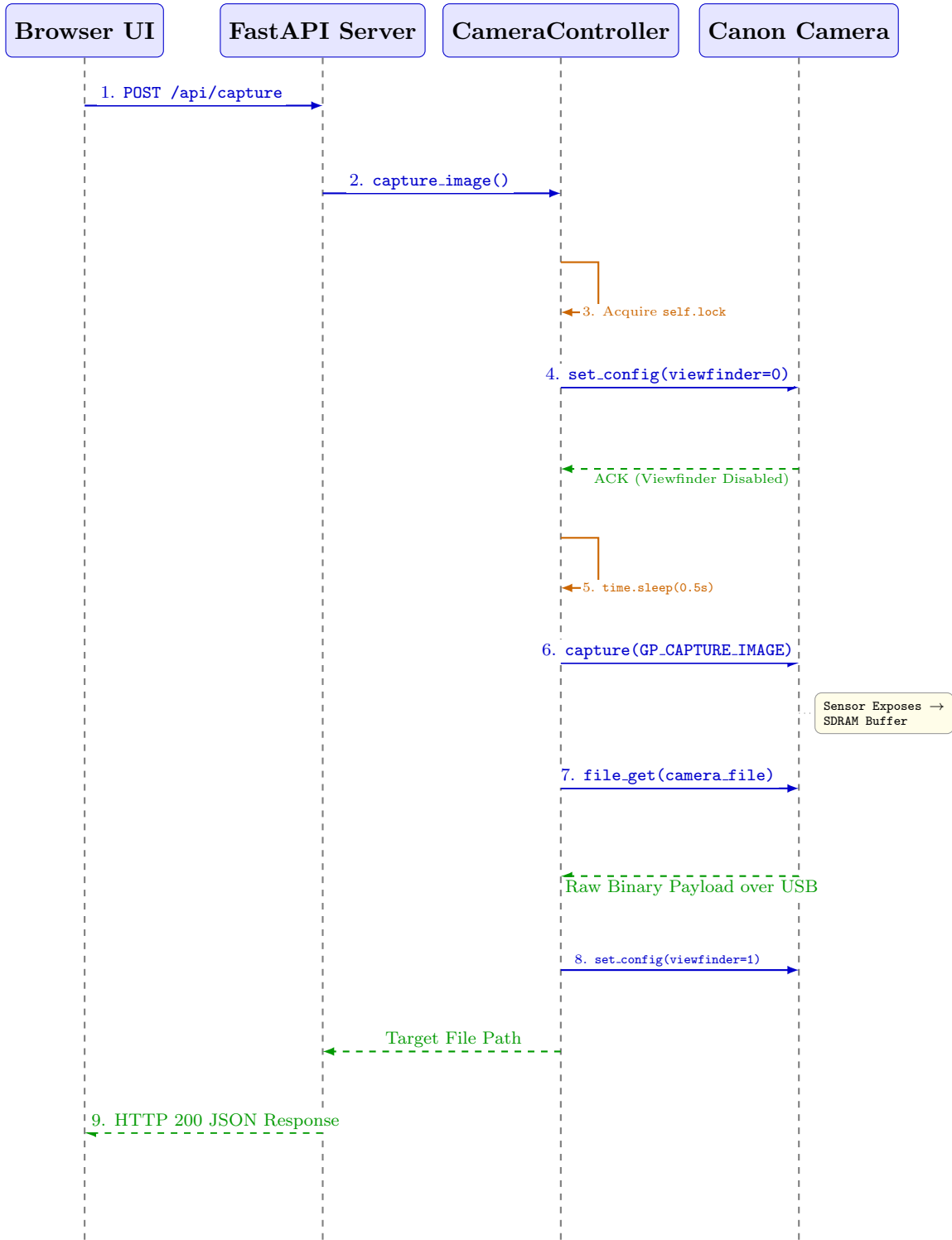


Figure 3: UML Sequence Diagram with boundary-connected lifelines and offset note box.

1.4 PTP Data Extraction and Preview Mechanics

1.4.1 PTP Capture Target and In-Memory Storage

Upon connection initialization, the controller sets `capturetarget = 1`. This directs the DIGIC X processor to store image data directly in volatile camera SDRAM buffer space rather than flushing to physical SD card memory, preventing file system locks.

1.4.2 C-Python Buffer Pointer Retrieval

The binary transfer is executed cleanly without file system overhead:

```
1 # Allocate raw C memory file container wrapper
2 camera_file = gp.CameraFile()
3
4 # Fetch binary buffer directly over USB Bulk IN Endpoint
5 self.camera.file_get(file_path.folder, file_path.name,
6                       gp.GP_FILE_TYPE_NORMAL, camera_file, self.
7                       context)
8
9 # Flush binary data buffer to target path
10 camera_file.save(target_path)
```

1.4.3 Live Preview Frame Extraction

For real-time streaming, the backend continuously pulls preview frames from the viewfinder buffer memory via `capture_preview()`:

```
1 def capture_preview(self):
2     with self.lock:
3         camera_file = gp.CameraFile()
4         self.camera.capture_preview(camera_file, self.context)
5         file_data = camera_file.get_data_and_size()
6         return memoryview(file_data).tobytes()
```

The FastAPI generator packages these frame buffers into an HTTP stream (`multipart/x-mixed-replace; boundary=frame`) refreshed at ≈ 20 FPS.